C++ Programming language Cheatsheet

4 years ago

C++ is a widely used middle-level programming language which is used in developing major operating systems(Windows, Linux, Android, Ubuntu, iOS etc), Games, databases and compilers etc.

Basics

- `cin >> x` : read value into the variable x from input stream
- `cout << x` : printf value to the output stream
- `//` : single line comments
- `/* */` : Multi line comments

```
#include <iostream>
using namespace std;
int main() {
    cout << "Hello World!!";
    return 0;
}
```

- `#include <iostream>` : iostream is a inbuilt header library which allows you to deal with input and output objects like cout etc.
- `using namespace std` : Specifies that the object and variable names can be used from standard library.
- `cin` : to accept input from standard input device i.e keyboard.
- `cout` : to print the output.
- `main()` : main function is the entry point of any C++ program.
- `return 0` : To end the main function

How to compile a program in C++

Open your terminal, Navigate to the directory where you have saved your program. Consider firstprogram.cpp is the name of your program.

```
sudo g++ -o firstprogram firstprogram.cpp
```

How to run a C++ program

```
./firstprogram
```

Data types

Types	Data-type
Basic	int, char, float, double, short, short int, long int etc
Derived	array, pointer, string, etc
User Defined Data Type	structure, enum, Class, Union, Typedef

Variables

```
data-type variable-name = value;
```

```
int value = 10; // declaring int variable and assigning value 10 to it
char grade = 'A'; // declaring char variable and assigning value A to it
```

Naming convention

- only letters (both uppercase and lowercase letters), digits and underscore(`_`).
- cannot contain white spaces
- First letter should be either a letter or an underscore(`_`).
- Variable type can't be changed

- Case sensitive
- Keywords cannot be used as variable names

Arrays

```
data-type array-name[size]; // one-dimensional Array
data-type array-name[size][size]; // two-dimensional Array
```

Example:

```
int a[5] = {1,2,3,4,5};
int a[2][3] = {
                {1,2,3},
                {4,5,6}
              };
```

String

It stores a sequence of character and it also functions like a vector.
It contain many fuctions:
begin(),end(),sort(),length() and many more

Example:

```
string s="abcde";                   // string declaration
cout<<s[2];                         // prints c
sort(s.begin(),s.end());            // sorts the given string
s.length();                         // returns the length of
the string
#include <string>                   // Include string (std
namespace)
s1.size(), s2.size();               // Number of characters: 0,
5
s1 += s2 + ' ' + "world";           // Concatenation
s1 == "hello world"                 // Comparison, also <, >,
!=, etc.
s1.substr(m, n);                    // Substring of size n
starting at s1[m]
s1.c_str();                         // Convert to const char*
```

```
s1 = to_string(12.05);          // Converts number to
string
getline(cin, s);                // Read line ending in '\n'
string string1.append(string2); // It is used to
concatenate two strings(string1 and string2 are two string
names)
```

Literals or Constants

Literal	Example
Integer Literal- decimal	255
Integer Literal- octal	0377
Integer Literal- hexadecimal	0xFF
Integer Literal- int	30
Integer Literal- unsigned int	30u
Integer Literal- long	30l
Integer Literal- unsigned long	30ul
Float point Literal	53.0f, 79.02
Character literals	'a', '1'
String literals	"OneCompiler", "Foo"
Boolean literals	true, false

Escape sequences

Escape sequence	Description
\n	New line
\r	Carriage Return
?	Question mark
\t	Horizontal tab

\v	Vertical tab
\f	Form feed
\	Backslash
'	Single quotation
"	Double quotation
\0	Null character
\b	Back space
\a	Audible Bell
\ooo	Octal
\xhhh	Hexadecimal

Operators

Operator type	Description
Arithmetic Operator	+ , - , * , / , %
comparision Operator	< , > , <= , >=, != , ==
Bitwise Operator	& , `|` , ^ , >> , << , ~
Logical Operator	&& , `||` , !
Assignment Operator	= , += , -= , *= , /= , %=, <<=, >>=, &=, ^=, `|=`
Ternary Operator	? :
sizeof operator	sizeof()

Scope Resolution Operator	:: (used to reference the global variable or member function that is out of scope.)

Operator Precedence and Associativity

Category	Operator	Associativity
Postfix	() [] -> . ++ - -	Left to right
Unary	+ - ! ~ ++ - - (type)* & sizeof	Right to left
Multiplicative	* / %	Left to right
Additive	+ -	Left to right
Shift	<< >>	Left to right
Relational	< <= > >=	Left to right

Keywords(reserved words)

```
auto        double      int         struct
break       else        long        switch
case        enum        register    typedef
char        extern      return      union
const       float       short       unsigned
continue    for         signed      void
default     goto        sizeof      volatile
do          if          static      while
try         catch       throw       asm
operator    new         template    this
public      private     protected   inline
alignas     namespace   template    bool
```

Conditional Statements

1. If

```
if(conditional-expression)
{
    //code
}
```

2. If-else

```
if(conditional-expression)
{
    //code
} else {
    //code
}
```

If-else using Ternary Operator

```
conditional-expression ? code1 : code2;
```

if conditional-expression is true, code1 is executed.
And, if condition is false, code2 is executed.

3. If-else-if ladder

```
if(conditional-expression-1)
{
    //code
} else if(conditional-expression-2) {
    //code
} else if(conditional-expression-3) {
    //code
}
....
else {
    //code
}
```

4. Switch

```
switch(conditional-expression){
case value1:
```

```
 //code
 break;  //optional
case value2:
 //code
 break;  //optional
...

default:
 //code to be executed when all the above cases are not
matched;
}
```

Loops

1. For

```
for(Initialization; Condition; Increment/decrement){
//code
}
```

2. While

```
while(condition){
//code
}
```

3. Do-While

```
do{
//code
} while(condition);
```

4. Range based for loop

```
double prices[5] = {4.99, 10.99, 6.87, 7.99, 8.49};
for (double x : prices)
cout << x << std::endl;
```

Functions

Function is a sub-routine which contains set of statements.

```
// declaring a function
return_type function_name(parameters);

// defining a function
return_type function_name(parameters){
//code
}

// calling a function
function_name (parameters)
```

Pointers

Pointer is a variable which holds the memory information(address) of another variable of same data type.

```
datatype *pointername;
(or)
datatype* pointername;
```

Example

```
int x = 10, *ptr;

/*ptr = x; // Error because ptr is adress and x is value
*ptr = &x;  // Error because x is adress and ptr is value
*/

ptr = &x; // valid because &x and ptr are addresses
*ptr = x; // valid because both x and *ptr values
```

'this' keyword

The this keyword is used to refer to that instance of the class which is under

function for the present situation.

Example

```
class student
{
    student(int marks)
    this->marks=marks;
    void print()
    cout<<this->marks;  //this will print marks of the
called instance of the class
}
```

References

When a vairable is declared as a reference , it becomes an alternative name for an existing vairable.

```
datatype& reference_name;
```

Example

```
#include <iostream>
using namespace std;

int main(){

    int x = 10;
    cout<< x ;   // Output is 10.

    int& xref = x;   // xref is a reference to x.

    xref = 20;  // value of x is modified to 20 using xref
reference.
    cout<< x ;  // Output is 20 ;

    return 0;
}
```

Structures

Structure is a user-defined data type where it allows you to combine data of different data types.

```
struct structure_name {

   member definition;
   member definition;
   ...
   member definition;
} [one or more structure variables];

struct structure_name variable name; //declaring structure
variables
```

Classes

A class in C++ is the building block that leads to Object-Oriented programming. It is a user-defined data type, which holds its own data members and member functions, which can be accessed and used by creating an instance of that class. A C++ class is like a blueprint for an object.

```
class MyClass {       // The class
  public:             // Access specifier
    int myNum;        // Attribute (int variable)
    string myString;  // Attribute (string variable)
};
```

Enum

Enumeration data type is a user-defined data type in C++. `enum` keyword is used to declare a new enumeration types in C++.

```
enum name{constant1, constant2, constant3, ..... } var-
list;
```

Example

```
enum month{January, February, March, April, May, June,
July, August, September, October, November, December} name;
```

Typedef

Typedef is used to explicitly define new data type names by using the keyword `typedef`. It defines a name for an existing data type but doesn't create a new data type.

```
typedef data-type name;
```

Example

```
typedef unsigned int distance;  // typedef of int
```

Macros

Macro is defined by #define directive. Whenever a macro name is encountered by the compiler, it replaces the name with the definition of the macro. Macro definitions need not be terminated by a semi-colon(;).

Example

```
//Can be writtern before or after
#inculde<iostream>
//but should be before the main()
```

```
#define ll long long //ll is then refrenced as long long in
the whole code
```

- `#define` : this keyword lets the compiler to understand the meaning of the keyword writtern next to it like in the example **ll** whenever written in code automatically expands as **long long**.

Genrally, used in Competetive coding and in big projects where need to

write same function is repetetive

Vectors

Vectors are same as dynamic arrays. They will be resized on element inseertions & deletions.

Functions associated with the vector are:

Following functions return iterator
begin()
end()
rbegin()
rend()
cbegin()
cend()
crbegin()
crend()

```
#include <iostream>
#include <vector>

using namespace std;

int main()
{
    vector<int> v1;

    for (int i = 1; i <= 10; i++)
        v1.push_back(i);

    cout << "Output of begin and end: ";
    for (auto i = v1.begin(); i != v1.end(); ++i)
        cout << *i << " ";

    return 0;
}
```

If we want to initialize 2d vectors or even 3d vectors (i.e. arrays) then we need to embed the vectors inside the vector data structures accordingly i.e.

if we want a 2d vector then vector is embedded inside a vector data structure like

```
vector<vector<int>> v2;//
```

or we can for 3d vector like

```
vector<vector<vector<int>>> v3;
```

Here are some useful vector functions you might need with short descriptions of each of them:

Title	Description
size()	Returns the size of the vector (1-based indexing)
front()	Returns the first element of the vector
back()	Returns the last element of the vector(Useful when you don't know the vector size)
push_back()	Add element at the end of vector
pop_back()	Delete element from the end of vector
insert()	Insert any element at any position of your choice (You need to pass the position first as an iterator and then the element)
erase()	Remove elements from vector (You can either pass a single iterator denoting the position of the element to be erased or two iterators denoting the range in which you want all elements to be erased)
empty()	Returns whether the container is empty(equivalent to size()==0 but faster)

Stacks

Stacks are a type of container adaptors with LIFO(Last In First Out) type of working, where a new element is added at one end (top) and an element is removed from that end only.

The functions associated with stack are:
empty() – Returns whether the stack is empty – Time Complexity : O(1)
size() – Returns the size of the stack – Time Complexity : O(1)
top() – Returns a reference to the top most element of the stack – Time Complexity : O(1)
push(g) – Adds the element 'g' at the top of the stack – Time Complexity : O(1)
pop() – Deletes the top most element of the stack – Time Complexity : O(1)

```
#include <iostream>
#include <stack>
using namespace std;
int main() {
        stack<int> stack;
        stack.push(21);
        stack.push(22);
        stack.push(24);
        stack.push(25);

                stack.pop();
        stack.pop();

        while (!stack.empty()) {
                cout << ' ' << stack.top();
                stack.pop();
        }
}
```

Queue

Queues are a type of container adaptors that operate in a first in first out (FIFO) type of arrangement. Elements are inserted at the back (end) and are deleted from the front

queue::empty() Returns whether the queue is empty.
queue::size() Returns the size of the queue.
queue::swap() Exchange the contents of two queues but the queues must be of the same type, although sizes may differ.
queue::emplace() Insert a new element into the queue container, the new element is added to the end of the queue.
queue::front() Returns a reference to the first element of the queue.
queue::back() Returns a reference to the last element of the queue.
queue::push(g) Adds the element 'g' at the end of the queue.
queue::pop() removes the element

```
#include <iostream>
#include <queue>
using namespace std;
int main() {
        queue<int> q;
        q.push(21);
        q.push(22);
        q.push(24);
        q.push(25);

                q.pop();
        q.pop();

        while (!q.empty()) {
                cout << ' ' << q.front();
                q.pop();
        }
}
```

Heap

Heap data structure is a complete binary tree that satisfies the **heap property**, where any given node is

- Always greater than its child node/s and the key of the root node is the largest among all other nodes. This property is also called **max heap property**.
- Always smaller than the child node/s and the key of the root node is the smallest among all other nodes. This property is also called **min**

heap property.

Different types of heap:

1. Max Heap

When a element inserted in heap wil be inserted in such position that greatest element will always be at top and will pe popped first.

```
#include<bits/stdc++.h>
using namespace std;
int main(){
    priority_queue<int,vector<int>> maxh;
    maxh.push(1);
    maxh.push(2);
    maxh.push(-1);

    int greatest = maxh.top(); // greatest = 2
    pq.pop();
    int second_greatest = maxh.top(); // second_greatest =
1
    return 0;
}
```

2. Min Heap

When a element inserted in heap wil be inserted in such position that smallest element will always be at top and will pe popped first.

```
#include<bits/stdc++.h>
using namespace std;
int main(){
    priority_queue<int,vector<int>,greater<int>> minh;
    minh.push(1);
    minh.push(2);
    minh.push(-1);

    int smallest = minh.top(); // smallest = -1
    pq.pop();
    int second_minimum = minh.top(); // second_minimum = 1
    return 0;
```

```
}
```

Hash Map

Hash Map (also, hash table) is a data structure that basically maps keys to values. A hash table uses a hash function to compute an index into an array of buckets or slots, from which the corresponding value can be found.
It can be implemented

```
unordered_map<int,int> mp; //here first element is the key and second element is the value
```

The first element is the key where the second element which is the value is stored and can be serached for in constant time

```
#include<bits/stdc++.h>
using namespace std;
int main(){
    unordered_map<int,int> mp; //
    mp.insert({1,2}); // key=1, value=2;
    mp[-1]=1; // key=-1, value=1
    return 0;
}
```

There can be an ordered map too which stores the keys in order

```
#include<bits/stdc++.h>
using namespace std;
int main(){
    map<int,int> order_mp; //
    order_mp.insert({1,2}); // key=1, value=2;
    order_mp[-1]=1; // key=-1, value=1
    return 0;
}
```

Sort one-line

Sorting is one of the most basic functions applied to data. It means arranging the data in a particular fashion, which can be increasing or

decreasing. There is a builtin function in C++ STL by the name of sort(). This function internally uses IntroSort. In more details it is implemented using hybrid of QuickSort, HeapSort and InsertionSort.

```
#include<iostream>
#include<algorithm>
using namespace std;

int main(){

int arr[] = {3,2,1};
int n = 3 ; // size of the array

sort(arr, arr+n); // sorting the array in ascending order
sort(arr, arr+n, greater<int>()); // sorting the array in
descending order

vector<int> v;

v = {3,2,1};

sort(v.begin(), v.end()); // sorting in the vector in
ascending order
sort(v.begin(), v.end(), greater<int>()); // sorting in the
vector in descending order
//For sorting the vector in descending order, you could
also use the reverse iterator
//i.e.:
sort(v.rbegin(),v.rend()); // This will also sort the
vector in a descending order

//for sorting only a part of the vector , we can do:
//sort(v.begin()+start_index,v.begin()+ending_index);
//This will sort only part of the array from start_index to
end_index

}
```

Header Files

Name	Use

#include<stdio.h>	It is used to perform input and output operations
#include<string.h>	It is used to perform various string operations
#include<math.h>	It is used to perform mathematical operations
#include<iomanip.h>	It is used to access set() and setprecision()
#include<signal.h>	It is used to perform signal handling functions like signal() and raise()
#include<stdarg.h>	It is used to perform standard argument functions
#include<errno.h>	It is used to perform error handling operations like errno
#include<fstream.h>	It is used to control the data to read from a file
#incl	

ude<time.h>	It is used to perform functions related to date() and time
#include<graphics.h>	It is used include and facilitate graphical operations in program
#include<bits/stdc++.h>	It is used to include all the standard library files, this is called a NON-STANDARD header file (it can't be compiled on a compiler other than GCC)
#include<limits.h>	The macros in this header stores the values of variables in various data types
#include<setjmp.h>	It contains function prototypes for functions that allow bypassing of the normal function call and return sequence
#include<assert.h>	It contains information for adding diagnostics that helps in program debugging
#include<float.h>	It contains a set of various platform-dependent constants related to floating point values
#include<ctype.h>	It contains function prototypes for functions that test characters for certain properties , and also function prototypes for functions that can be used to convert uppercase letters to lowercase letters and vice versa
#incl	

ude<iostream>	It provides basic input and output services for C++ programs

Java
C++
Groovy
Tcl
CommonLisp
Erlang
Assembly
Python2
Ruby
Racket
Basic
Bootstrap
Bulma
Skeleton
BackboneJS
Vue (Beta)
Clojure
Kotlin
Rust
Octave
CoffeeScript
Oracle Database
SQLite
Cassandra

Python
NodeJS
JShell
Lua
D
F#
Scala
C#
Go
OCaml
HTML
JQuery
Uikit
Milligram
React (Beta)
Vue3 (Beta)
TypeScript
Pascal
Swift
Text
EJS
PostgreSQL
Redis
Oracle PL/SQL

C
JavaScript
Haskell
Ada
Elixir
Fortran
PHP
Perl
R
Visual Basic (VB.NET)
Materialize
Foundation
Semantic UI
PaperCSS
Angular (Beta)
Bash
Cobol
Prolog
Objective-C
BrainFK
MySQL
MongoDB
MariaDB
Microsoft SQL Server

More

Orgs
API
Pricing

Cheatsheets
Tutorials
Tools
Stats